

AI Customer Support Triage Platform

Overview

This project is a full-stack customer support ticket system with built-in triage, agent workflows, ML-powered categorization and priority prediction, and optional automation.

It includes:

- a backend API in `backend/` using ASP.NET Core and .NET 10
- a frontend UI in `frontend/` using Angular 19
- AI/ML support for ticket classification and priority prediction
- automation and background jobs via Hangfire when enabled
- a SQL Server-ready data model via Entity Framework Core

The platform supports three roles:

- `customer`
- `agent`
- `admin`

Each role has distinct pages, APIs, and access controls.

Tech Stack

Backend

- .NET 10
- ASP.NET Core Web API
- Entity Framework Core
- Microsoft.EntityFrameworkCore.SqlServer
- Serilog
- FluentValidation
- Hangfire (optional)
- ML.NET
- BCrypt for password hashing
- JWT-style auth

Frontend

- Angular 19
- Angular Router
- Angular Signals
- HttpClient
- Angular guards
- ng2-charts / Chart.js
- Auth0 Angular JWT for token-handling assistance

Database

- MS SQL Server is the intended database provider
- EF Core model maps all entities and relationships
- `backend/appsettings.json` contains the connection string and feature flags

AI / ML

- ML.NET FastTree models for category classification and priority prediction
- fallback keyword/rule-based logic when ML is unavailable
- optional AI reply suggestion through OpenRouter / Gemini

Backend Architecture

`backend/Program.cs`

This file starts the backend and configures all major services.

Key responsibilities:

- configure Serilog logging
- read feature flags and configuration
- set up SQL Server if `Features:UseSqlServer` is true
- set up Hangfire if `Features:UseHangfire` is true
- register controllers, memory cache, HTTP client, and validators
- set CORS policy for the frontend
- enforce bearer token auth via middleware
- create the upload folder
- optionally ensure the SQL database is created and seeded
- map controllers to routes
- optionally expose Hangfire dashboard
- run the app on `http://localhost:5000`

`backend/appsettings.json`

Contains runtime configuration:

- SQL Server connection string
- `Features:UseSqlServer`
- `Features:UseHangfire`
- upload storage path
- JWT secret and expiration

Example use:

- enable SQL Server after a local instance is running
- enable Hangfire if you want recurring job execution

Data layer

`backend/Data/AppDbContext.cs`

This file defines the Entity Framework Core data model.

It declares `DbSet<>` properties for:

- User
- Team
- TeamMember
- Category
- Ticket
- TicketComment
- TicketAttachment
- TicketHistoryEntry

It also configures:

- unique constraints (like user email)
- max field lengths
- required fields
- foreign key relationships
- delete behavior
- SQL Server `datetime2` columns
- default values such as `GETUTCDATE()`

backend/Services/Store.cs

The project also contains an in-memory runtime store.

This store:

- keeps all runtime data in `ConcurrentDictionary`s
- seeds initial users, teams, categories, and membership
- supports ID generation for tickets, comments, attachments, history, and teams

Important note: most controllers currently use this `Store` rather than EF Core directly.

That means the app is architected for SQL Server, but the active data path is still an in-memory prototype.

Backend Services and Helpers

backend/Services/Auth.cs

Handles authentication and authorization.

Provides:

- current user lookup from bearer token
- role enforcement helpers:
 - `RequireRole`
 - `RequireAnyRole`
 - `RequireTicketAccess`
- JWT-style token issuing and validation
- password hashing and verification

The `Tokens` helper creates HS256 tokens and validates them on every request.

backend/Services/Api.cs

Standardizes API responses.

Provides:

- validation error formatting
- `Unauthorized`, `Forbidden`, `NotFound` utilities
- mapping methods to convert domain objects into response DTOs
- ticket list and ticket detail shapes
- comment DTO conversion

backend/Services/TicketHistoryService.cs

Records ticket history entries whenever a ticket field changes.

Each history entry captures:

- ticket ID
- changed field
- old value
- new value
- user who made the change
- timestamp

Controllers (API surface)

backend/Controllers/AuthController.cs

Auth endpoints:

- `POST /api/auth/register`
 - creates a new customer account
 - hashes password with `BCrypt`
- `POST /api/auth/login`
 - verifies credentials
 - returns auth token and user info
- `GET /api/auth/me`
 - returns the authenticated user

backend/Controllers/TicketsController.cs

Ticket lifecycle endpoints:

- `POST /api/tickets`
 - customers submit new tickets
 - ticket status is set to `New`
 - a ticket classification job is enqueued
- `GET /api/tickets`

- returns paginated ticket lists
 - filters by role, status, priority, and search
- GET /api/tickets/{id}
 - returns ticket detail with comments, attachments, and history
- PUT /api/tickets/{id}/status
 - allows agents/admins to change status
- PUT /api/tickets/{id}/priority
 - change priority and update SLA
- PUT /api/tickets/{id}/category
 - assign or reassign category
- PUT /api/tickets/{id}/assign
 - admin-only assignment to team/agent
- POST /api/tickets/{id}/ai-reply
 - agents/admins can request an AI suggested reply

backend/Controllers/DashboardController.cs

Dashboard endpoints:

- GET /api/dashboard/stats
 - returns ticket metrics for agents and admins
 - includes counts, SLA compliance, category breakdown, priority breakdown, and daily ticket volume
 - results are cached for 30 seconds
- GET /api/dashboard/agents
 - admin-only agent workload metrics
 - includes open tickets, resolved today, and average resolution time

backend/Controllers/CategoriesController.cs

Category management endpoints:

- GET categories
- POST create category
- PUT update category
- DELETE category

Admin-only access is enforced for write operations.

backend/Controllers/TeamsController.cs

Team management endpoints:

- GET teams
- POST create team
- POST add team member
- PUT update team
- DELETE team

Admin-only access is enforced for write operations.

backend/Controllers/CommentsController.cs

Comment endpoints:

- GET /api/tickets/{ticketId}/comments
- POST /api/tickets/{ticketId}/comments

Support for internal comments is available for agents/admins.
Customers can only see public comments.

backend/Controllers/AttachmentsController.cs

Attachment endpoints:

- POST upload attachment to a ticket
- GET download attachment by ID

The backend validates:

- file size (5MB limit)
- file type/extensions
- ticket access
- attachment count per ticket (up to 3)

AI and ML

backend/Services/MLService.cs

This service provides ticket intelligence.

It supports:

- category classification
- priority prediction

It uses ML.NET FastTree models when available and falls back to rules when ML is not available.

Category classification

- first attempts ML model prediction
- if ML fails, uses keyword matching
- returns category name and confidence score

Priority prediction

- first attempts ML model prediction
- if ML fails, uses keyword and content rules

- returns priority and confidence score

Model files

Models are stored in `backend/ML/`:

- `category-fasttree-model.zip`
- `priority-fasttree-model.zip`
- `training_data.csv`
- `priority_training_data.csv`

If the model files are missing or training data changes, the service retrains them automatically.

AI Suggested Reply

The backend also includes `AiReplyService.cs`.

This service can:

- call an external AI provider via OpenRouter
- generate a reply suggestion for a ticket
- keep the API key hidden from the frontend

The frontend calls `/api/tickets/{id}/ai-reply` to retrieve suggested responses.

Automation and Background Jobs

`backend/Services/TicketJobDispatcher.cs`

Dispatches ticket-related jobs.

It supports two modes:

- Hangfire queue mode when `Features.UseHangfire` is enabled
- in-process background execution otherwise

Job types:

- classify a ticket
- predict ticket priority
- assign a ticket

`backend/Jobs/*.cs`

Job implementations include:

- `AssignTicketJob.cs`
 - assigns a ticket to an agent and team
- `ClassifyTicketJob.cs`
 - classifies ticket category
- `PredictPriorityJob.cs`
 - predicts ticket priority
- `AutoCloseJob.cs`
 - closes resolved tickets automatically on a schedule
- `RetrainModelsJob.cs`
 - retrains ML models on a schedule

Hangfire

When enabled, Hangfire provides:

- background execution
- recurring jobs
- a dashboard at `/hangfire`

It uses SQL Server storage when enabled.

Database and SQL Server

Intent

The project is intended to use MS SQL Server as the database.

Evidence:

- `backend/Program.cs` registers `AppDbContext` with `UseSqlServer`
- the project depends on `Microsoft.EntityFrameworkCore.SqlServer`
- `backend/appsettings.json` contains a SQL Server connection string
- Hangfire storage is also configured for SQL Server

Current runtime behavior

Important detail:

- the data model and SQL Server wiring exist
- most controllers still use the in-memory `Store` in `backend/Services/Store.cs`

So while SQL Server is the planned database provider, the current implementation is still using in-memory storage for active ticket operations.

How to enable SQL Server

In `backend/appsettings.json`, set:

```
"Features": {
  "UseSqlServer": true,
  "UseHangfire": true
}
```

Then ensure SQL Server is available at the configured connection string.

Database schema

The EF Core schema includes entities for:

- users
- teams
- team members
- categories
- tickets
- ticket comments
- ticket attachments
- ticket history entries

Relationships are configured with foreign keys and restricted deletes.

Frontend Architecture

Angular project structure

Key folders:

- `src/app/pages/` — page components for each route
- `src/app/services/` — API service wrappers and shared logic
- `src/app/guards/` — authentication and role guards
- `src/app/models/` — DTO shapes and type definitions
- `src/app/app.routes.ts` — route definitions and access rules

Route map and pages

The frontend supports role-specific routes:

Customer routes

- `/customer/dashboard`
- `/customer/tickets/new`
- `/customer/tickets/:id`

Agent routes

- `/agent/dashboard`
- `/agent/tickets`
- `/agent/tickets/:id`
- `/agent/sla-alerts`
- `/agent/reports`

Admin routes

- `/admin/dashboard`
- `/admin/review-queue`
- `/admin/teams`
- `/admin/tickets`
- `/admin/tickets/:id`
- `/admin/categories`
- `/admin/agents`
- `/admin/reports`
- `/admin/sla-monitoring`
- `/admin/settings`

Public routes

- `/login`
- `/register`

Route guarding is implemented with:

- `AuthGuard` — blocks unauthenticated users
- `roleGuard(role)` — blocks users without the required role

Core frontend services

AuthService

Handles authentication state and login flow.

It:

- calls `/api/auth/register` and `/api/auth/login`
- stores token and user info in `localStorage`
- exposes login state and the current user via signals
- navigates users to `{role}/dashboard`

TicketService

Wraps ticket-related backend APIs.

It supports:

- list tickets with filters
- get ticket detail
- create tickets
- upload attachments
- post comments
- request AI suggested reply
- change ticket status, priority, category
- assign ticket to agent/team

DashboardService

Wraps dashboard APIs:

- `GET /api/dashboard/stats`
- `GET /api/dashboard/agents`

Frontend UI flows

Customer experience

- register and login

- create support tickets
- view ticket status and history
- attach files
- read ticket comments

Agent experience

- view assigned ticket dashboard
- review ticket SLAs and priorities
- open ticket details
- add comments and internal notes
- update status, priority, category
- request AI reply suggestions

Admin experience

- monitor overall dashboard metrics
- manage teams and categories
- inspect review queue
- assign tickets
- manage agents
- view reports and SLA monitoring

AI and ML Details

Classification and prediction flow

When a ticket is created:

- the backend enqueues a classification job
- ML tries to predict the category from the ticket text
- ML also tries to predict priority when needed
- if ML is unavailable, rule-based logic applies

ML model files

Relevant files are in backend/ML/:

- training_data.csv
- priority_training_data.csv
- category-fasttree-model.zip
- priority-fasttree-model.zip

Fallback rules

When ML fails to load or predict, the backend uses keyword and simple heuristics:

- category keywords such as refund, error, password, feature, contact
- priority urgency words like urgent, asap, emergency
- category and message length influence priority

AI suggested replies

The backend supports external AI reply generation via `AiReplyService.cs`.

This is separate from the ML models and can use an external API provider.

The frontend calls `/api/tickets/{id}/ai-reply` for an AI-generated reply suggestion.

Automation and Background Jobs

Job flow

Ticket automation is dispatched by `TicketJobDispatcher`:

- classification job
- priority prediction job
- assignment job

If Hangfire is disabled, jobs still execute in-process asynchronously.

Recurring jobs

When Hangfire is enabled, recurring jobs are created in `Program.cs`:

- auto-close resolved tickets every hour
- retrain ML models every month

Why this matters

Automation reduces manual work by:

- assigning tickets automatically
- predicting ticket category and priority
- keeping SLA deadlines updated
- closing tickets automatically when criteria are met
- refreshing ML models periodically

Database and SQL Server

Intended database provider

The project is designed for MS SQL Server.

In backend/`Program.cs`:

- `builder.Services.AddDbContext<AppDbContext>(options => options.UseSqlServer(...))`
- `builder.Services.AddHangfire(options => options.UseSqlServerStorage(...))`

In backend/`appsettings.json`:

- SQL Server connection string for localhost, 1433
- feature toggles to turn SQL Server on/off

Current data flow caution

Currently, the API controllers use the in-memory `Store` for most data operations.
That means:

- the backend can run without SQL Server
- the current ticket data is stored in memory while the app runs
- the SQL Server model is present, but not fully wired into every controller path yet

How to make the DB active

To use SQL Server persistently, set:

```
"UseSqlServer": true,  
"UseHangfire": true
```

in `backend/appsettings.json` and run SQL Server locally.

Database schema

Key tables and entities:

- Users
- Teams
- TeamMembers
- Categories
- Tickets
- TicketComments
- TicketAttachments
- TicketHistoryEntries

Relationships:

- Ticket.CreatedByUserId → Users
- Ticket.AssignedAgentId → Users
- Ticket.AssignedTeamId → Teams
- Ticket.CategoryId → Categories
- TicketComment.TicketId → Tickets
- TicketAttachment.TicketId → Tickets
- TicketHistoryEntry.TicketId → Tickets

Entity constraints

The EF Core model enforces:

- unique user email
- unique team and category names
- required titles, descriptions, statuses
- max string lengths
- default timestamps
- restricted delete behavior for data integrity

Running the project

Backend

From the root:

```
dotnet run --project backend\Backend.csproj
```

Open:

- API root: `http://localhost:5000`
- Swagger UI: `http://localhost:5000/swagger`
- Hangfire dashboard: `http://localhost:5000/hangfire` if enabled

Frontend

From frontend:

```
npm start
```

Open:

- `http://localhost:4200/login`

Optional SQL Server + Hangfire

Use Docker to run SQL Server and then enable features in `backend/appsettings.json`.

How to use this document to answer questions

This document is intended to let you answer questions about:

- the overall system architecture
- frontend vs backend responsibilities
- authentication and role access
- ticket workflow and ticket state management
- ML and AI capabilities
- automation and scheduled jobs
- database intent, configuration, and current runtime behavior

If someone asks:

- "What does the backend do?" — point to the Backend Architecture section.
- "How are tickets classified?" — point to AI and ML section.
- "What database is used?" — point to Database and SQL Server section.
- "How does the frontend communicate with the backend?" — point to Frontend Architecture and Core services sections.

Project file map

Backend

- `Program.cs` — app startup and configuration
- `appsettings.json` — runtime config and feature flags
- `Data/AppDbContext.cs` — EF Core data model
- `Controllers/` — API endpoints

- `Services/` — auth, store, ML, API helpers, job dispatch
- `Jobs/` — automated background jobs
- `ML/` — model training data and model files

Frontend

- `src/app/app.routes.ts` — route definitions
 - `src/app/guards/` — auth and role guards
 - `src/app/services/` — API service wrappers
 - `src/app/models/` — type definitions
 - `src/app/pages/` — route components
 - `src/environments/environment.ts` — backend API URL
-

Summary

This project is a full-stack prototype for an AI-enabled support ticket system.

Main capabilities:

- customer ticket submission
- agent and admin dashboards
- ticket comments and attachments
- ML-based category and priority prediction
- AI suggested replies
- ticket assignment and SLA automation
- SQL Server-ready architecture

This document should give you enough context to explain what each part does and how the project works end to end.